

HDOC – Experiment 35

Question

Write a menu-driven C program to read a variable-length square integer array and perform the following operations using separate functions:

1. Determine whether all the elements on its main diagonal are distinct.
2. Find the sum of its main diagonal elements.
3. Display its elements using a diagonal traversal. Begin with the element in the top-left corner and traverse successive diagonals alternately in the upward-right and downward-left directions.
4. Exit.

❖ Algorithm

1. Start.
2. Read the order n of the square matrix.
3. If $n \leq 0$, display an error message and ask again.
4. Declare a variable-length integer array $a[n][n]$.
5. Read all matrix elements.
6. Repeatedly display the menu:
 - 1 – Check whether all main diagonal elements are distinct.
 - 2 – Find the sum of main diagonal elements.
 - 3 – Display the matrix using diagonal traversal.
 - 4 – Exit.
7. For **Option 1**:

- Compare every main diagonal element $a[i][i]$ with the diagonal elements following it.
- If any two are equal, the diagonal elements are not distinct.
- Otherwise, they are distinct.

8. For **Option 2**:

- Initialize $sum = 0$.
- Add $a[i][i]$ for $i = 0$ to $n-1$.
- Display the sum.

9. For **Option 3**:

- Start from the top-left element.
- Process diagonals according to the constant value of $row + column$.
- Traverse successive diagonals alternately:
 - upward-right,
 - downward-left.
- Display each element in traversal order.

10. For 4, terminate the program.

11. For any other choice, display an invalid-choice message.

12. Stop.

❖ Program code in C

```
#include <stdio.h>
```

```
void checkDistinct(int n, int a[n][n]);
```

```
void diagonalSum(int n, int a[n][n]);
```

```
void diagonalTraversal(int n, int a[n][n]);
```

```
int main()
{
    int n, i, j, choice;

    /* Read and validate the order of the square matrix */
    do
    {
        printf("Enter the order of the square matrix: ");
        scanf("%d", &n);

        if (n <= 0)
            printf("Invalid input. Order must be positive.\n");
    } while (n <= 0);

    /* Variable-Length 2D Array */
    int a[n][n];

    printf("Enter the matrix elements:\n");

    for (i = 0; i < n; i++)
    {
        for (j = 0; j < n; j++)
        {
            scanf("%d", &a[i][j]);
        }
    }

    do
```

```
{  
    printf("\n--- MENU ---\n");  
    printf("1. Check whether main diagonal elements are distinct\n");  
    printf("2. Find sum of main diagonal elements\n");  
    printf("3. Diagonal traversal\n");  
    printf("4. Exit\n");  
  
    printf("Enter your choice: ");  
    scanf("%d", &choice);  
  
    switch (choice)  
    {  
        case 1:  
            checkDistinct(n, a);  
            break;  
  
        case 2:  
            diagonalSum(n, a);  
            break;  
  
        case 3:  
            diagonalTraversal(n, a);  
            break;  
  
        case 4:  
            printf("Exiting the program.\n");  
            break;
```

```
        default:

            printf("Invalid choice. Please enter a choice from 1 to 4.\n");

        }

    } while (choice != 4);

    return 0;
}
```

```
/* Function to check whether main diagonal elements are distinct */
```

```
void checkDistinct(int n, int a[n][n])
```

```
{
```

```
    int i, j;
```

```
    int distinct = 1;
```

```
    printf("Main diagonal elements: ");
```

```
    for (i = 0; i < n; i++)
```

```
    {
```

```
        printf("%d ", a[i][i]);
```

```
    }
```

```
    printf("\n");
```

```
    for (i = 0; i < n - 1; i++)
```

```
    {
```

```
        for (j = i + 1; j < n; j++)
```

```

    {
        if (a[i][i] == a[j][j])
        {
            distinct = 0;
            break;
        }
    }

    if (distinct == 0)
        break;
}

if (distinct)
    printf("All the main diagonal elements are distinct.\n");
else
    printf("All the main diagonal elements are not distinct.\n");
}

```

/* Function to find sum of main diagonal elements */

```
void diagonalSum(int n, int a[n][n])
```

```

{
    int i, sum = 0;

    for (i = 0; i < n; i++)
    {
        sum += a[i][i];
    }
}

```

```
printf("Sum of main diagonal elements: %d\n", sum);  
}
```

```
/* Function to perform diagonal traversal */
```

```
void diagonalTraversal(int n, int a[n][n])
```

```
{
```

```
    int d, i, j;
```

```
    printf("Diagonal traversal: ");
```

```
    /*
```

```
        d represents the diagonal number.
```

```
        For every element in one diagonal:
```

```
        row + column = d
```

```
    */
```

```
    for (d = 0; d <= 2 * n - 2; d++)
```

```
    {
```

```
        if (d % 2 == 0)
```

```
        {
```

```
            /*
```

```
                Even-numbered diagonal:
```

```
                traverse upward-right.
```

```
            */
```

```
            if (d < n)
```

```

        i = d;
    else
        i = n - 1;

    j = d - i;

    while (i >= 0 && j < n)
    {
        printf("%d ", a[i][j]);

        i--;
        j++;
    }
}

else
{
    /*
        Odd-numbered diagonal:
        traverse downward-left.
    */

    if (d < n)
        j = d;
    else
        j = n - 1;

    i = d - j;

    while (j >= 0 && i < n)

```

```

    {
        printf("%d ", a[i][j]);

        i++;

        j--;
    }

}

printf("\n");
}

```

❖ Test case table

Test Case	Matrix / Input	Operation	Expected Result
1	n = 3, matrix 1 2 3 / 4 5 6 / 7 8 9	Distinct diagonal	1 5 9 are distinct
2	Same matrix	Diagonal sum	15
3	Same matrix	Diagonal traversal	1 2 4 7 5 3 6 8 9
4	1 2 3 / 4 1 6 / 7 8 9	Distinct diagonal	Not distinct because diagonal is 1 1 9
5	n = 1, matrix 5	All operations	Distinct; sum = 5; traversal = 5
6	n = 0	Matrix size	Invalid input; asks again
7	Valid matrix, choice 6	Menu	Invalid choice message
8	Valid matrix, choice 4	Exit	Program terminates

❖ Compile, Execute and Evaluate

```
(kali㉿kali)-[~/Desktop/exp35]
$ vi array8.c

(kali㉿kali)-[~/Desktop/exp35]
$ gcc array8.c -o array8

(kali㉿kali)-[~/Desktop/exp35]
$ ./array8
Enter the order of the square matrix: 3
Enter the matrix elements:
1 2 3
4 5 6
7 8 9

— MENU —
1. Check whether main diagonal elements are distinct
2. Find sum of main diagonal elements
3. Diagonal traversal
4. Exit
Enter your choice: 1
Main diagonal elements: 1 5 9
All the main diagonal elements are distinct.

— MENU —
1. Check whether main diagonal elements are distinct
2. Find sum of main diagonal elements
3. Diagonal traversal
4. Exit
Enter your choice: 2
Sum of main diagonal elements: 15

— MENU —
1. Check whether main diagonal elements are distinct
2. Find sum of main diagonal elements
3. Diagonal traversal
4. Exit
Enter your choice: 3
Diagonal traversal: 1 2 4 7 5 3 6 8 9
```

— MENU —

1. Check whether main diagonal elements are distinct
2. Find sum of main diagonal elements
3. Diagonal traversal
4. Exit

Enter your choice: 4

Exiting the program.

(kali@kali)-[~/Desktop/exp35]

\$./array8

Enter the order of the square matrix: 3

Enter the matrix elements:

1 2 3

4 1 6

7 8 9

— MENU —

1. Check whether main diagonal elements are distinct
2. Find sum of main diagonal elements
3. Diagonal traversal
4. Exit

Enter your choice: 1

Main diagonal elements: 1 1 9

All the main diagonal elements are not distinct.

— MENU —

1. Check whether main diagonal elements are distinct
2. Find sum of main diagonal elements
3. Diagonal traversal
4. Exit

Enter your choice: 4

Exiting the program.

(kali@kali)-[~/Desktop/exp35]

\$./array8

Enter the order of the square matrix: 1

Enter the matrix elements:

5

— MENU —

1. Check whether main diagonal elements are distinct
2. Find sum of main diagonal elements
3. Diagonal traversal
4. Exit

Enter your choice: 1

Main diagonal elements: 5

All the main diagonal elements are distinct.

— MENU —

1. Check whether main diagonal elements are distinct
2. Find sum of main diagonal elements
3. Diagonal traversal
4. Exit

Enter your choice: 2

Sum of main diagonal elements: 5

— MENU —

1. Check whether main diagonal elements are distinct
2. Find sum of main diagonal elements
3. Diagonal traversal
4. Exit

Enter your choice: 3

Diagonal traversal: 5

— MENU —

1. Check whether main diagonal elements are distinct
2. Find sum of main diagonal elements
3. Diagonal traversal
4. Exit

Enter your choice: 4

Exiting the program.

(kali@kali)-[~/Desktop/exp35]

\$./array8

Enter the order of the square matrix: 0

Invalid input. Order must be positive.

Enter the order of the square matrix: 1

Enter the matrix elements:

```

Invalid input. Order must be positive.
Enter the order of the square matrix: 1
Enter the matrix elements:
6
— MENU —
1. Check whether main diagonal elements are distinct
2. Find sum of main diagonal elements
3. Diagonal traversal
4. Exit
Enter your choice: 1
Main diagonal elements: 6
All the main diagonal elements are distinct.

— MENU —
1. Check whether main diagonal elements are distinct
2. Find sum of main diagonal elements
3. Diagonal traversal
4. Exit
Enter your choice: 4
Exiting the program.

(kali㉿kali)-[~/Desktop/exp35]
$ ./array8
Enter the order of the square matrix: 2
Enter the matrix elements:
1 3
4 2
— MENU —
1. Check whether main diagonal elements are distinct
2. Find sum of main diagonal elements
3. Diagonal traversal
4. Exit
Enter your choice: 6
Invalid choice. Please enter a choice from 1 to 4.

— MENU —
1. Check whether main diagonal elements are distinct

```

```
— MENU —  
1. Check whether main diagonal elements are distinct  
2. Find sum of main diagonal elements  
3. Diagonal traversal  
4. Exit  
Enter your choice: 4  
Exiting the program.
```

```
(kali㉿kali)-[~/Desktop/exp35]  
$
```